Conciliador Financiero

TP/MVP Demonstration Guide

Purpose: A shareable, evidence-based guide for reviewing and demonstrating the current academic financial reconciliation MVP.

Current release posture: The deployed happy path is verified through login, two valid CSV imports, pending records, proposal generation, one proposal confirmation, and history recording. Remote reversal smoke verification remains **pending**.

Audience: Teammates, reviewers, and professors evaluating the TP/MVP.

Public deployment

- **?** Frontend: https://conciliador-frontend-8djd.onrender.com
- **?** API health check: https://conciliador-api-bym3.onrender.com/health
- **?** Render Blueprint: `conciliador-financiero-mvp`
- **?** Fixture repository: https://github.com/turkaym/conciliador-financiero/tree/feat/final-mvp/docs/segunda-entrega/csv/fixtures

Demo data warning: The credentials in this guide are fictional demo credentials. Do not use them, or the shared demo database, with real financial data.

1. What This Guide Proves

The guide gives a reviewer one reproducible route through the current deployed MVP:

- **?** Sign in with the demo account.
- **?** Import valid movement and receipt CSV files.
- **?** Confirm that each file reports 3/3 valid rows.
- **?** Inspect six pending records and three generated proposals.
- **?** Confirm Proposal #1 manually.
- **?** Verify the `PROPUESTA_CONFIRMADA` event in **Historial**.
- **?** Treat reversal as a clearly marked follow-up test, not as completed evidence.

The product assists reconciliation; it does not replace the human decision to confirm or reject a proposal.

2. The Problem the TP Solves

Companies and accounting teams may receive bank movements and receipts from separate sources. Comparing those files manually is repetitive, makes exceptions easy to miss, and can leave little evidence of why a pair was accepted or rejected.

Conciliador addresses the academic version of that problem by bringing two known CSV sources into one review flow. It validates the input, preserves pending records, proposes explainable pairs, requires a human decision, and records the resulting event history.

This is a project hypothesis and an executable MVP demonstration, not a claim that business value or production suitability has already been proven through interviews, metrics, or real customer data.

3. MVP Scope

Included

Capability	Current behavior
Authentication	JWT demo login for a pre-registered user
Import	Manual CSV upload using known source and template catalogs
Validation	Structure and row validation with visible errors
Data scope	Fictional ARS movements and receipts
Matching	One movement to one receipt, subject to explicit eligibility rules
Review	Human confirmation or rejection of generated proposals
Pending work	List and inspect unresolved records
Traceability	History of imports, decisions, and reconciliation events
Deployment	React/Vite frontend and FastAPI API deployed on Render

Excluded or not yet demonstrated

- ? Real bank, ERP, AFIP, or accounting-system integrations.
- ? OCR, artificial intelligence, predictive matching, or autonomous confirmation.
- ? Multi-currency reconciliation, expenses, and 1:N or N:M relationships.
- ? Multi-tenancy, advanced roles, administration, dashboards, exports, and advanced analytics.
- ? A production security posture or suitability for real financial data.
- ? Remote reversal completion evidence. The reversal endpoint and workflow are implemented, but the remote smoke test is still pending.

4. Architecture and Data Flow

The MVP uses a modular monolith with a clear web-to-database path:

```
React/Vite/TypeScript UI
|
FastAPI/Python API and JWT authentication
|
Application and domain rules
|
SQLAlchemy persistence
|
PostgreSQL
```

The main flow is:

- ? The operator logs in and receives a demo JWT.
- ? The UI loads compatible source and template options.
- ? A CSV is uploaded and assigned to an import batch.
- ? The backend validates structure and rows, preserving original values and recording errors.
- ? Valid records become pending records. Invalid rows do not prevent other valid rows from being processed.
- ? A completed batch with valid rows triggers proposal generation against opposite historical pending records.
- ? The operator reviews and confirms or rejects a proposal.
- ? History records the relevant actor, target, state, and event.

The deployed frontend is available at https://conciliador-frontend-8djd.onrender.com. The deployed API is https://conciliador-api-bym3.onrender.com. Render manages the deployment through the `conciliador-financiero-mvp` Blueprint.

5. Business Matching Rules

A movement and receipt are eligible only when all three mandatory conditions hold:

Rule	Requirement
Currency	Both records use ARS.
Amount	The amounts are exactly equal.
Date	The business dates are within an inclusive +/- 3 calendar-day window.

Reference text is not an eligibility gate. It is normalized and used to rank eligible candidates and explain the ordering. The current normalizer is `ref-nfkd-v1`: it applies Unicode NFKD, removes marks, converts to uppercase, replaces punctuation with spaces, collapses whitespace, trims, and uses unique tokens. Candidate ordering prioritizes exact reference equality, reference similarity, smaller date difference, and stable IDs.

No proposal is confirmed automatically. A person must select **Confirmar propuesta** or **Rechazar propuesta**. Rejection requires a non-empty reason.

6. Implementation Status and Evidence

Verified in the deployed flow

- ? The public frontend loads and accepts the demo login.
- ? `movimientos-validos.csv` imports with 3/3 valid rows.
- ? `comprobantes-validos.csv` imports with 3/3 valid rows.
- ? The resulting review state contains six pending records.
- ? The system generates three proposals.
- ? **Proposal #1** is confirmed successfully through the UI.
- ? **Historial** contains the `PROPUESTA_CONFIRMADA` event.

Pending verification

- ? Remote reversal smoke test: use **Revertir conciliación** with a reason, then verify the result and any re-generated proposals. Do not present this as completed evidence until it has been observed remotely.

Local implementation checks

At the latest implementation checkpoint, the project recorded 35 backend tests, 13 frontend tests, a TypeScript check, a production build, and Compose/configuration checks. These are implementation checks, not a claim that every final release gate or remote scenario has passed.

7. Teammate Test: Exact Steps

Before starting

- ? Use a normal browser window and the public frontend URL.
- ? Use only the approved fictional fixtures.
- ? Keep the API health URL available for a cold-start check.
- ? The shared demo database may already contain prior test data.

Step 1: Login

- ? Open https://conciliador-frontend-8djd.onrender.com.
- ? Enter email `demo@conciliador.local`.
- ? Enter password `Demo1234!`.
- ? Submit the form.
- ? Confirm that the authenticated application shell appears.

These credentials are demo-only and must not be used for real data.

Step 2: Import movements

- ? Open **Importar**.
- ? Choose the movement source and its compatible template.
- ? Select `movimientos-validos.csv` from the approved fixture repository.
- ? Submit the import.
- ? Confirm a successful processed result with 3/3 valid rows and no row errors.

Step 3: Import receipts

- ? Stay in **Importar**.
- ? Choose the receipt source and its compatible template.
- ? Select `comprobantes-validos.csv` from the same fixture repository.
- ? Submit the import.
- ? Confirm a successful processed result with 3/3 valid rows and no row errors.

Step 4: Inspect pending records

- ? Open **Pendientes**.
- ? Confirm that the two imports produce six pending records in total.
- ? Check that the records retain their source type, date, amount, currency, and original reference where present.

Step 5: Review proposals

- ? Open **Propuestas**.
- ? Confirm that three proposals are listed.
- ? Open Proposal #1.
- ? Review both sides of the proposed pair and the displayed explanation.
- ? Confirm that eligibility is consistent with ARS, exact amount, and date within +/- 3 days.
- ? Treat reference similarity as ranking evidence, not as the reason eligibility exists.

Step 6: Confirm one proposal

- ? Select **Confirmar propuesta** for Proposal #1.
- ? Confirm the successful response in the UI.
- ? Check that the proposal is no longer awaiting the same decision and that the reconciliation is active.

Step 7: Verify history

- ? Open **Historial**.
- ? Find the event `PROPUESTA_CONFIRMADA`.

? Verify that the event is associated with the confirmation just performed.

Step 8: Reversal follow-up, pending

? Locate the active reconciliation created by the confirmation.

? Select **Revertir conciliación**.

? Enter a non-empty reason.

? Record the observed response, resulting states, history event, and any generated proposals.

? Mark the test as **verified only after remote observation**. Until then, report it as pending.

8. Test Fixtures and Expected Results

All approved fixtures are under the repository path `docs/segunda-entrega/csv/fixtures`.

File	Expected result
`movimientos-validos.csv`	Processed; 3 valid movement rows; 3 pending records before matching.
`comprobantes-validos.csv`	Processed; 3 valid receipt rows; 3 pending records before matching.
`movimientos-invalidos.csv`	`PROCESADO_SIN_VALIDOS`; 3 invalid rows; no proposals.
`comprobantes-invalidos.csv`	`PROCESADO_SIN_VALIDOS`; 3 invalid rows; no proposals.
`movimientos-duplicados.csv`	`PROCESADO_CON_ERRORES`; 2 eligible rows and 2 invalid duplicate rows.
`comprobantes-duplicados.csv`	`PROCESADO_CON_ERRORES`; 2 eligible rows and 2 invalid duplicate rows.
`movimientos-estructura-invalida.csv`	`FALLIDO`; no processed rows; incompatible-header error.
`comprobantes-estructura-invalida.csv`	`FALLIDO`; no processed rows; incompatible-header error.

The two valid files are the canonical happy-path test for the deployed flow. Invalid fixtures demonstrate validation behavior and should be run only when the reviewer wants to inspect error handling.

9. Known Limitations and Troubleshooting

Render cold start

The first request after inactivity can be slow while a Render service wakes up. Wait, refresh once, and check https://conciliador-api-bym3.onrender.com/health. A healthy response is expected to report `{"status":"healthy"}`.

A valid file is reported as duplicate

The shared demo database may already contain the same test data. Re-uploading the exact same valid file bytes can therefore be reported as a duplicate. This is an honest property of the shared environment, not evidence that the fixture is invalid. Use the existing records for review, or coordinate a clean test state rather than repeatedly uploading the same bytes.

Import result differs from the expected count

Check that the correct fixture, source type, and compatible template were selected. Confirm that the browser sent the intended file and inspect the import result errors before retrying.

No proposals appear

Confirm that both imports completed with valid rows, that the currency is `ARS`, that amounts match exactly, and that dates are within three days. A `PROCESADO_SIN_VALIDOS`, `FALLIDO`, or `DUPLICADO` batch does not trigger matching.

Reporting an error

Record the frontend URL, approximate time, exact steps, selected fixture, visible status or error code, and a screenshot that contains no real data. Report the issue through the repository Issues page. Never include secrets, tokens, or real financial records.

10. Short Presentation Script

“Conciliador is an academic MVP that assists reconciliation between two known CSV sources. I log in with a demo account, import fictional movements and receipts, and show that validation produces three valid rows per file. The system leaves six records pending and generates three proposals. Its mandatory eligibility rules are ARS, exact amount, and a date difference of at most three days; reference similarity only ranks eligible candidates. I then review Proposal #1, confirm it manually, and show the `PROPUESTA_CONFIRMADA` event in Historial. Remote reversal verification is still pending, so I will not claim that step as completed.”

Questions a professor may ask

Question	Concise answer
Why is confirmation manual?	The MVP assists the reviewer but does not replace a business decision.
Why PostgreSQL?	The flow needs relational integrity, 1:1 constraints, transactions, and traceability.
Does a similar reference make a match eligible?	No. It only ranks candidates after currency, amount, and date pass.
What happens to invalid rows?	They receive validation errors while other valid rows can continue processing.
Is this production-ready?	No. It is an academic MVP using fictional data and demo credentials.
What remains open?	Remote reversal smoke verification and broader product validation.
What is deliberately out of scope?	Real integrations, OCR/AI, multitenancy, multi-currency, advanced analytics, and autonomous decisions.

11. Final Status Checklist

Verified

- **?** [x] Public frontend loads.
- **?** [x] Demo login succeeds.
- **?** [x] Valid movement CSV imports 3/3 valid.
- **?** [x] Valid receipt CSV imports 3/3 valid.
- **?** [x] Six pending records are visible.
- **?** [x] Three proposals are visible.
- **?** [x] Proposal #1 confirmation succeeds.
- **?** [x] `PROPUESTA_CONFIRMADA` appears in **Historial**.
- **?** [x] Local implementation checks were recorded at the latest checkpoint.

Pending

- **?** [ ] Remote **Revertir conciliación** smoke test is observed end to end.
- **?** [ ] Reversal result, history event, and any re-generated proposals are recorded as evidence.

Out of scope

- **?** [ ] Real financial data or production credentials.
- **?** [ ] Real bank, ERP, AFIP, or accounting integrations.
- **?** [ ] OCR, AI, predictive matching, or automatic confirmation.
- **?** [ ] Multi-currency, expenses, 1:N/N:M reconciliation, multitenancy, and advanced administration.
- **?** [ ] SLA, performance guarantee, or final release-gate claim based only on the local checkpoint.

Status conclusion: The current TP/MVP supports a verified deployed happy path for import, pending review, proposal confirmation, and history. Reversal remains an explicitly pending verification item.